# Parallelization of morphology-based image processing

Samuele Marrani
samuele.marrani@edu.unifi.it

## Abstract

*Morphological Image Processing focuses on digital image processing techniques based on **mathematical morphology**, a fundamental approach in image analysis. This approach relies on **structuring elements (SE)**, small matrices applied to each pixel of an image, where the new pixel value is computed as the minimum or maximum of its neighboring pixels defined by the structuring element. This approach enables common operations such as erosion, dilation, opening, closing, and gradient extraction.*
*The main goal of this project is to compare the computational performance of three different implementations of morphological image processing: a **sequential C++** implementation, a parallel CPU version using **OpenMP**, and a GPU-based implementation using **CUDA**. The comparison highlights the impact of parallelism on execution time and efficiency across different hardware architectures.*

**Future Distribution Permission**
The author(s) of this report give permission for this document to be distributed to Unifi-affiliated students taking future courses.

## 1 Introduction

In image processing, *mathematical morphology* is a technique based on the use of *structuring elements*, small matrices that define the neighborhood of each pixel and are used to probe and transform digital images. The application of a structuring element to an image is performed through morphological operations, in which each output pixel is computed as the minimum or maximum of neighboring input pixels covered by the structuring element. Morphological processing represents a fundamental approach in digital image analysis and is widely used for operations such as noise removal, image segmentation, and edge extraction. These methods operate directly in the spatial domain and form the basis of many classical computer vision algorithms. Morphological image processing is particularly well suited for parallel computation, since these operations are applied independently to each pixel. This makes it an ideal case study for evaluating different computational paradigms, ranging from sequential execution on a single CPU core to parallel implementations on multi-core CPUs and GPUs.

### 1.1 Mathematical formulation

From an algorithmic perspective, morphological image processing is based on neighborhood operations defined by a structuring element. Let $I(x, y)$ denote the input image and $S(i, j)$ a structuring element of size $(2m+1)\times(2n+1)$, centered at position $(0, 0)$. The output image $O(x, y)$ is obtained by sliding the structuring element across the image and computing, for each pixel, the minimum or maximum of the corresponding neighborhood:

$$O(x,y) = \begin{cases} \min\limits_{(i,j)\in S} I(x+i,\, y+j) & \text{(erosion)} \\ \max\limits_{(i,j)\in S} I(x+i,\, y+j) & \text{(dilation)} \end{cases}$$

This operation consists of evaluating the pixel values within the neighborhood defined by the structuring element and selecting either the minimum or maximum value, depending on the operation. These operations can be interpreted as non-linear filtering techniques in the spatial domain. In practical implementations, special care must be taken when processing image boundaries, since the structuring element can partially extend

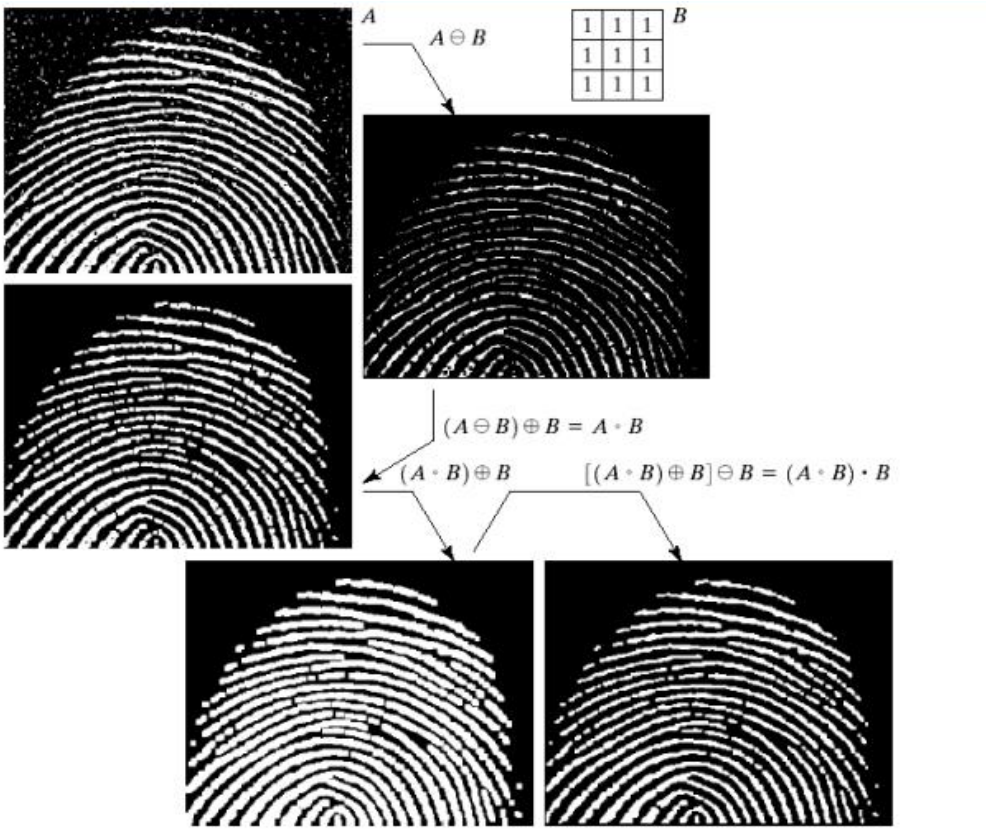


Figure 1: Morphological image processing schema.

beyond the image limits. Common strategies include zero padding, border replication, or mirroring. Due to its regular memory access pattern and high degree of data parallelism, morphological operations are particularly suitable for optimization on parallel architectures such as CPUs with OpenMP and GPUs with CUDA.

### 1.2 Morphological operations

In this section, we present a set of discrete morphological operations commonly employed for shape analysis, noise removal, and structure enhancement in binary and grayscale images. Each operation is defined with respect to a finite structuring element $S$ with odd spatial support $n \in \{3, 5, 7\}$, applied to an input image $I$ through standard morphological transformations, as implemented in our C++ / CUDA framework.

**Dilation** Dilation is a morphological operation designed to expand bright regions of an image. Given a structuring element $S$, dilation replaces each pixel with the maximum value within its neighborhood defined by $S$. This operation enlarges object boundaries, fills small holes, and connects nearby structures. From a signal processing perspective, dilation can be interpreted as a nonlinear maximum filter, emphasizing dominant local intensities while suppressing small dark variations.

**Erosion** Erosion performs the dual operation of dilation by shrinking bright regions. Each pixel is replaced with the minimum value within the neighborhood defined by the structuring element $S$. This process removes small-scale bright noise, separates weakly connected components, and reduces object boundaries. Erosion can be interpreted as a nonlinear minimum filter, attenuating local maxima and preserving darker structures.

(a) Original image (b) Processed image

Figure 2: Gradient result (1024×1024 image with $5 \times 5$ structuring element).

**Opening** Opening is defined as the sequential application of erosion followed by dilation using the same structuring element. This operation removes small objects and smooths contours without significantly altering the overall shape of larger structures. It is particularly effective for eliminating isolated noise while preserving the main geometric features of the image. From a morphological standpoint, opening acts as a size-selective filter that suppresses structures smaller than the structuring element.

**Closing** Closing is the dual operation of opening and consists of dilation followed by erosion. It is used to fill small holes, bridge narrow gaps, and smooth object boundaries. Unlike opening, closing preserves bright regions while eliminating small dark artifacts. This operation is especially useful when the goal is to enhance the continuity of structures without introducing significant distortions.

**Morphological Gradient** The morphological gradient is an operation designed to extract the boundaries of objects by computing the difference between dilation and erosion of the same image with respect to a structuring element $S$. Formally, it can be expressed as

$$G(I) = \delta_S(I) - \varepsilon_S(I),$$

where $\delta_S$ and $\varepsilon_S$ denote dilation and erosion, respectively. This operation highlights regions of sharp intensity transition, corresponding to object contours, while suppressing homogeneous areas. From a signal processing perspective, the morphological gradient can be interpreted as a nonlinear edge detector that captures local intensity variations without relying on linear approximations of derivatives. Compared to standard gradient-based methods, it provides better localization of edges and increased robustness to noise, especially when combined with appropriately chosen structuring elements. An example is shown in Figure 2.

**Structuring Element Considerations** Increasing the size of the structuring element from $3 \times 3$ to $7 \times 7$ generally enhances the ability to remove noise and enforce spatial coherence, at the cost of reduced sensitivity to fine details and increased computational complexity. The choice of structuring element size therefore represents a trade-off between detail preservation and morphological robustness, depending on the target application and performance constraints.

### 1.3 OpenMP

OpenMP (Open Multi-Processing) is an application programming interface for parallel programming on shared-memory architectures, widely adopted on modern multi-core systems. It enables an incremental parallelization approach, allowing developers to extend sequential programs written in C, C++, or Fortran through compiler directives, runtime library routines, and environment variables. OpenMP follows a *fork–join* execution model, where a master thread spawns a team of worker threads to execute parallel regions and synchronizes with them at the end of each region. The programmer can explicitly control work sharing, synchronization, and data scoping, achieving a balance between ease of use and expressive power.

### 1.4 CUDA

CUDA (Compute Unified Device Architecture) is a parallel computing platform and programming model developed by NVIDIA for general-purpose computing on graphics processing units (GPUs). CUDA adopts a heterogeneous execution model in which the CPU (host) manages the execution of massively parallel kernels on the GPU (device).
CUDA exposes a hierarchical execution model: kernels are executed by a large number of lightweight threads organized into thread blocks and grids. This hierarchy maps directly onto the underlying GPU hardware and enables fine-grained data parallelism at scale. The CUDA *memory model* is explicit and hierarchical, reflecting the diverse memory spaces available on the GPU. These include *global memory*, which is accessible by all threads but has relatively high latency; *shared memory*, which is shared among threads within the same block and offers low-latency access; *constant memory*, a specialized, read-only memory space; and per-thread private memory such as registers and local memory. Efficient utilization of these memory spaces is critical for achieving high performance and requires careful management by the programmer.

## 2 Proposed solutions

In this project, several sequential and parallel solutions are proposed to address the *Morphological-based image processing* problem. All of them are written in the C++ programming language and some of them also use CUDA C dialect. These approaches consist of the following versions:

1. Sequential;
2. Parallel OMP;
3. Parallel CUDA.

### 2.1 Common Implementation Aspects

All approaches share a common design in terms of data representation and processing workflow, ensuring a fair comparison across different execution models. The processing pipeline consists of the following main stages:

1. grayscale image acquisition or loading from .pgm file;
2. structuring element creation
3. morphological processing, during which structuring elements are applied to the image data;
4. execution time measuring
5. processed image saving to .pgm file.

Images are represented as one-dimensional vectors of floating-point values in row-major order to optimize memory access locality and pixel intensities are normalized within the range [0, 255]. Structuring elements are defined as square matrices with odd dimensions (e.g., 3×3, 5×5, 7x7). Structuring element values belong to the binary set $\{0, 1\}$, which represent the activation or not of the morphological operation on that pixel. Their memory layout is adapted to the access patterns of each execution model.
To show implementation details, the *erosion* operation will be taken as the example.

## 2.2 Sequential Implementation

The sequential implementation performs the operation by iterating over each input image pixel and computing the minimum (in case of erosion; in case of dilation it's the maximum) of neighboring pixels pointed by the structuring element. The computation is organized using nested loops and follows a straightforward row-major memory access pattern. Each output pixel is processed independently and written to a separate buffer, ensuring correctness and simplicity.

```
for(int y = 0; y < img.getHeight(); y++) {
    for(int x = 0; x < img.getWidth();
        x++) {
        uint8_t min_val = 255;
        for(int j = 0; j < se.getSize();
            j++) {
            for(int i = 0; i <
                se.getSize(); i++) {
                if(se(i,j) == 0) continue;
                int xi = x + i - offset;
                int yj = y + j - offset;
                if(xi >=0 && xi <
                    img.getWidth() && yj
                    >=0 && yj <
                    img.getHeight()) {
                    min_val =
                        std::min(min_val,
                        img(xi,yj));
                }
            }
        }
        out(x,y) = min_val;
    }
}
```

## 2.3 OpenMP Implementation

The OpenMP-based parallel implementation extends the sequential approach by exploiting shared-memory parallelism. The outer loops of the operation process are parallelized using OpenMP directives, enabling multiple threads to process distinct image regions concurrently.

```
#pragma omp parallel for schedule(static)
for(int y = 0; y < img.getHeight(); y++) {
    for(int x = 0; x < img.getWidth();
        x++) {
        uint8_t min_val = 255;
        for(int j = 0; j < se.getSize();
            j++) {
            for(int i = 0; i <
                se.getSize(); i++) {
                if(se(i,j) == 0) continue;
                int xi = x + i - offset;
                int yj = y + j - offset;
                if(xi >=0 && xi <
                    img.getWidth() && yj
                    >=0 && yj <
                    img.getHeight()) {
                    min_val =
                        std::min(min_val,
                        img(xi,yj));
                }
            }
        }
        out(x,y) = min_val;
    }
}
```

Static scheduling is employed as the workload for each thread is uniform. Furthermore, using a dynamic approach would result in no benefit and also in higher overhead.
Since this project focuses primarily on the CUDA implementation, the choice of demanding the optimal thread count computing to the OpenMP library has been made. Furthermore, there can be the situation where threads share the same core when their number exceeds the number of physical cores. The automatic thread count approach also handles this situation.
No *collapse* directive has been used because it makes no sense to linearize two loops when threads are a lot less than the workload (as in this example, where threads are 10 times less than the pixels in the best case).
Lastly, thread safety is ensured by assigning independent output regions to each thread.

## 2.4 CUDA Implementation

The CUDA implementation takes advantage of GPU parallelism by applying three separate memory optimization strategies, each realized as an individual CUDA kernel. Fixed block dimensions of 16x16 are used and structuring elements up to 7x7 in size are supported (`MAX_SE_SIZE = 7`). Thread blocks are arranged in a 16×16 lay-

out, while grid dimensions are calculated dynamically based on the image size. The output values are naturally clamped as the initial maximum and minimum are set respectively to 255 and 0.

**Global Memory Implementation** The baseline implementation allocates and manages both the input image data and the structuring element in global memory using `cudaMalloc`/`cudaFree` and `cudaMemcpy`. This straightforward approach incurs higher memory access latency compared to shared or constant memory optimizations, as all threads must directly access global memory for both the image and structuring element data during morphological operations.

```
size_t img_bytes = width * height *
    sizeof(uint8_t);
size_t se_bytes = se_size * se_size *
    sizeof(uint8_t);
uint8_t *d_input, *d_output, *d_se;

cudaMalloc(&d_input, img_bytes);
cudaMalloc(&d_output, img_bytes);
cudaMalloc(&d_se, se_bytes);

cudaMemcpy(d_input, img.getDataR(),
    img_bytes, cudaMemcpyHostToDevice);
cudaMemcpy(d_se, se.getDataR(), se_bytes,
    cudaMemcpyHostToDevice);

dim3 block(BLOCK_SIZE, BLOCK_SIZE);
dim3 grid((width+BLOCK_SIZE-1)/BLOCK_SIZE,
    (height+BLOCK_SIZE-1)/BLOCK_SIZE);

// ...
erosion_global_kernel<<<grid,
    block>>>(d_input, d_output, width,
    height, d_se, se_size);
// ...

cudaDeviceSynchronize();
cudaMemcpy(output.getDataW(), d_output,
    img_bytes, cudaMemcpyDeviceToHost);

cudaFree(d_input);
cudaFree(d_output);
cudaFree(d_se);

return output;
```

Each thread computes its pixel coordinates using `blockIdx` and `threadIdx`, directly accessing global memory for both the input image and structuring element.

```
int x = blockIdx.x * blockDim.x +
    threadIdx.x;
int y = blockIdx.y * blockDim.y +
    threadIdx.y;
int offset = se_size / 2;

if(x >= width || y >= height) return;

uint8_t min_val = 255;
for(int j=0; j<se_size; j++){
    for(int i=0; i<se_size; i++){
        if(se[j*se_size + i] == 0)
            continue;
        int xi = x + i - offset;
        int yj = y + j - offset;
        if(xi >=0 && xi < width && yj >=0
            && yj < height){
            min_val = min(min_val,
                input[yj*width + xi]);
        }
    }
}
output[y*width + x] = min_val;
```

**Constant Memory Implementation** This implementation optimizes structuring element access by storing the it in constant memory using `cudaMemcpyToSymbol`.

```
__constant__ uint8_t
    d_se_const[MAX_SE_SIZE * MAX_SE_SIZE];
// ...

size_t img_bytes = width * height *
    sizeof(uint8_t);
uint8_t *d_input, *d_output;

cudaMalloc(&d_input, img_bytes);
cudaMalloc(&d_output, img_bytes);

cudaMemcpy(d_input, img.getDataR(),
    img_bytes, cudaMemcpyHostToDevice);
cudaMemcpyToSymbol(d_se_const,
    se.getDataR(), se_size*se_size *
    sizeof(uint8_t));

dim3 block(BLOCK_SIZE, BLOCK_SIZE);
dim3 grid((width+BLOCK_SIZE-1)/BLOCK_SIZE,
    (height+BLOCK_SIZE-1)/BLOCK_SIZE);

// ...
```

```
erosion_constant_kernel<<<grid,
    block>>>(d_input, d_output, width,
    height, se_size);
// ...

cudaDeviceSynchronize();
cudaMemcpy(output.getDataW(), d_output,
    img_bytes, cudaMemcpyDeviceToHost);

cudaFree(d_input);
cudaFree(d_output);

return output;
```

By exploiting the low-latency, read-only characteristics of constant memory, this approach reduces access overhead and improves overall kernel performance. While the image data remains in global memory, caching the structuring element in constant memory enhances access patterns when multiple threads concurrently read identical structuring element values.

```
int x = blockIdx.x * blockDim.x +
    threadIdx.x;
int y = blockIdx.y * blockDim.y +
    threadIdx.y;
int offset = se_size / 2;

if(x >= width || y >= height) return;

uint8_t min_val = 255;
for(int j=0; j<se_size; j++){
    for(int i=0; i<se_size; i++){
        if(d_se_const[j*se_size + i] == 0)
            continue;
        int xi = x + i - offset;
        int yj = y + j - offset;
        if(xi >=0 && xi < width && yj >=0
            && yj < height){
            min_val = min(min_val,
                input[yj*width + xi]);
        }
    }
}
output[y*width + x] = min_val;
```

**Shared Memory Implementation** The most advanced implementation leverages shared memory to construct a cached tile of the input image within each thread block.
This approach adopts a two-phase strategy: initially, threads cooperatively load a block of image data, including the necessary external pixels, into shared memory arrays and synchronize using `__syncthreads()`.
Subsequently, the morphological operation is executed utilizing the cached data. This methodology necessitates meticulous handling of boundary conditions and explicit thread synchronization, yet it substantially reduces the overhead associated with global memory accesses.

```
extern __shared__ uint8_t shared_mem[];
uint8_t* s_se = shared_mem;
uint8_t* s_img = shared_mem + se_size *
    se_size;

int tid = threadIdx.y * blockDim.x +
    threadIdx.x;
if(tid < se_size * se_size){
    s_se[tid] = se[tid];
}
__syncthreads();

int offset = se_size / 2;
int tx = threadIdx.x;
int ty = threadIdx.y;
int bx = blockIdx.x * blockDim.x;
int by = blockIdx.y * blockDim.y;

int x = bx + tx;
int y = by + ty;

int tile_width = BLOCK_SIZE + 2 * offset;
int tile_height = BLOCK_SIZE + 2 * offset;

int local_x = tx + offset;
int local_y = ty + offset;

for (int dy = ty; dy < tile_height; dy +=
    BLOCK_SIZE) {
    for (int dx = tx; dx < tile_width; dx
        += BLOCK_SIZE) {
        int gx = bx + dx - offset;
        int gy = by + dy - offset;

        if (gx >= 0 && gx < width && gy >=
            0 && gy < height) {
            s_img[dy * tile_width + dx] =
                input[gy * width + gx];
        } else {
            s_img[dy * tile_width + dx] =
                255;
        }
    }
}
__syncthreads();
```

```
if(x >= width || y >= height) return;

uint8_t min_val = 255;
for (int j = 0; j < se_size; j++) {
    for (int i = 0; i < se_size; i++) {
        if (s_se[j * se_size + i] == 0)
            continue;

        int sx = local_x + i - offset;
        int sy = local_y + j - offset;
        min_val = min(min_val, s_img[sy *
            tile_width + sx]);
    }
}
output[y * width + x] = min_val;
```

## 3 Experimental setup and results

This section describes the experimental setup and the experimental campaign conducted to evaluate the performance of the morphological processing application implemented using different computational paradigms. The application applies a morphological operation through a structuring element to an input image and measures the execution time of each implementation. To ensure a fair comparison, all implementations are evaluated under identical conditions.
The experimental analysis focuses exclusively on the *gradient* operation, as it represents the most complete and computationally heavy operation among those implemented.

The tests are performed on multiple sample images and by setting and varying the following parameters:

- **input image resolution**: $256 \times 256$, $512 \times 512$ and $1024 \times 1024$ pixels;
- **structuring element size**: $3 \times 3$, $5 \times 5$ and $7 \times 7$ pixels;
- **CUDA memory model** (only on CUDA version): global memory, constant memory and shared memory.
- **iterations**: 10 iterations by default, to minimize outlier misures.
- **cold start iterations**: 2 iterations by default, not to take into account cold start effects (especially when using CUDA).

For each test configuration, the input image is loaded from disk and the structuring element is generated according to the selected structuring element size. After the morphological operation, the resulting image is saved to disk and the execution time is measured using high-resolution timers.
All timings are collected in milliseconds and stored for subsequent analysis.

All experiments were conducted on a system with the following hardware and software specifications:

- CPU: Intel Core i5-7600 (4 cores - 4 threads, 3.50 GHz)
- GPU: NVIDIA GeForce GTX 1060 (1280 cores, 6GB)
- RAM: 16GB (2x8GB) DDR4-2400MHz
- Storage: 1 TB HDD
- Operating System: Linux Ubuntu 24.04

### 3.1 Sequential CPU implementation

The sequential CPU implementation serves as the baseline for performance comparison. The morphological operation is executed on a single CPU core by iterating over all image pixels and directly applying the structuring element, without any form of parallelism. For each image resolution and structuring element size, the execution time is measured by recording timestamps immediately before and after the morphological process. Upon completion, the resulting image is saved to disk. The collected execution times provide a reference point for evaluating the performance gains achieved by parallel CPU and GPU implementations.

### 3.2 OpenMP parallel implementation

The OpenMP implementation exploits shared-memory parallelism on the CPU by distributing the morphological processing workload across multiple threads. Since project focuses primarily on CUDA implementation, an approach of automatic thread count was chosen. This approach allows the evaluation of scalability with respect to the available CPU resources and demands the choice of optimal thread count to the OpenMP library. For each combination of image resolution and structuring element size the morphological operation is executed in parallel. Execution time is measured using the same methodology adopted for the sequential implementation, ensuring consistency across tests. Each run produces an output image and a corresponding timing record that includes all testing parameters, enabling a direct comparison of performance improvements relative to the baseline.

### 3.3 CUDA GPU implementation

The CUDA implementation offloads the morphological operation to the GPU, exploiting data-level parallelism. To evaluate the impact of different memory access strategies on performance, multiple CUDA memory configurations are tested. Specifically, three memory models are considered: global memory, constant memory and shared memory. For each memory configuration, the morphological operation is applied to images of varying resolutions and structuring element sizes. Before each execution, the availability of GPU resources is verified: if GPU isn't available, a fallback sequential CPU implementation is used. The execution time is measured using high-resolution timers, and the resulting image is saved to disk. Each test is recorded together with metadata describing the memory model employed and all other testing parameters. This experimental setup allows a detailed analysis of how different CUDA memory strategies influence performance when applying a computationally intensive gradient operation to an image.

| Gradient operation | | |
|---|---|---|
| S.E. | $3 \times 3$ | |
| Image size | $256 \times 256$ | |
| Sequential | 31.57 ms | |
| OpenMP | 26.75 ms | |
| | Configuration | Execution time [ms] |
| CUDA | Global memory | 0.82 |
| | Constant memory | 0.87 |
| | Shared memory | 1.01 |

Table 1: Execution times for the gradient operation using a $3 \times 3$ structuring element on a $256 \times 256$ image. Comparison between sequential CPU, OpenMP, and CUDA implementations.

| Gradient operation | | |
|---|---|---|
| S.E. | $5 \times 5$ | |
| Image size | $256 \times 256$ | |
| Sequential | 118.08 ms | |
| OpenMP | 36.21 ms | |
| | Configuration | Execution time [ms] |
| CUDA | Global memory | 0.89 |
| | Constant memory | 0.89 |
| | Shared memory | 0.93 |

Table 2: Execution times for the gradient operation using a $5 \times 5$ structuring element on a $256 \times 256$ image. Comparison between sequential CPU, OpenMP, and CUDA implementations.

| Gradient operation | | |
|---|---|---|
| S.E. | $7 \times 7$ | |
| Image size | $256 \times 256$ | |
| Sequential | 245.99 ms | |
| OpenMP | 51.99 ms | |
| | Configuration | Execution time [ms] |
| CUDA | Global memory | 0.95 |
| | Constant memory | 0.85 |
| | Shared memory | 0.70 |

Table 3: Execution times for the gradient operation using a $7 \times 7$ structuring element on a $256 \times 256$ image. Comparison between sequential CPU, OpenMP, and CUDA implementations.

| Gradient operation | | |
|---|---|---|
| S.E. | $3 \times 3$ | |
| Image size | $512 \times 512$ | |
| Sequential | 114.52 ms | |
| OpenMP | 29.36 ms | |
| | Configuration | Execution time [ms] |
| CUDA | Global memory | 1.15 |
| | Constant memory | 0.81 |
| | Shared memory | 0.85 |

Table 4: Execution times for the gradient operation using a $3 \times 3$ structuring element on a $512 \times 512$ image. Comparison between sequential CPU, OpenMP, and CUDA implementations.

| Gradient operation | | |
|---|---|---|
| S.E. | $5 \times 5$ | |
| Image size | $512 \times 512$ | |
| Sequential | 295.84 ms | |
| OpenMP | 83.09 ms | |
| | Configuration | Execution time [ms] |
| CUDA | Global memory | 1.22 |
| | Constant memory | 1.06 |
| | Shared memory | 0.92 |

Table 5: Execution times for the gradient operation using a $5 \times 5$ structuring element on a $512 \times 512$ image. Comparison between sequential CPU, OpenMP, and CUDA implementations.

| Gradient operation | | |
|---|---|---|
| S.E. | $7 \times 7$ | |
| Image size | $512 \times 512$ | |
| Sequential | 552.25 ms | |
| OpenMP | 150.46 ms | |
| | Configuration | Execution time [ms] |
| CUDA | Global memory | 1.64 |
| | Constant memory | 1.26 |
| | Shared memory | 1.10 |

Table 6: Execution times for the gradient operation using a $7 \times 7$ structuring element on a $512 \times 512$ image. Comparison between sequential CPU, OpenMP, and CUDA implementations.

| Gradient operation | | |
|---|---|---|
| S.E. | $3 \times 3$ | |
| Image size | $1024 \times 1024$ | |
| Sequential | 458.74 ms | |
| OpenMP | 126.15 ms | |
| | Configuration | Execution time [ms] |
| CUDA | Global memory | 2.44 |
| | Constant memory | 2.26 |
| | Shared memory | 2.43 |

Table 7: Execution times for the gradient operation using a $3 \times 3$ structuring element on a $1024 \times 1024$ image. Comparison between sequential CPU, OpenMP, and CUDA implementations.

| Gradient operation | | |
|---|---|---|
| S.E. | $5 \times 5$ | |
| Image size | $1024 \times 1024$ | |
| Sequential | 576.38 ms | |
| OpenMP | 160.81 ms | |
| | Configuration | Execution time [ms] |
| CUDA | Global memory | 2.73 |
| | Constant memory | 2.52 |
| | Shared memory | 2.41 |

Table 8: Execution times for the gradient operation using a $5 \times 5$ structuring element on a $1024 \times 1024$ image. Comparison between sequential CPU, OpenMP, and CUDA implementations.

| Gradient operation | | |
|---|---|---|
| S.E. | $7 \times 7$ | |
| Image size | $1024 \times 1024$ | |
| Sequential | 612.95 ms | |
| OpenMP | 178.53 ms | |
| | Configuration | Execution time [ms] |
| CUDA | Global memory | 3.81 |
| | Constant memory | 3.52 |
| | Shared memory | 3.21 |

Table 9: Execution times for the gradient operation using a $7 \times 7$ structuring element on a $1024 \times 1024$ image. Comparison between sequential CPU, OpenMP, and CUDA implementations.

### 3.4 Results summary

In this section we present a global summary of the execution times for the gradient operation, considering sequential and parallel implementations. Tables n.10, 11 and 12 show the sequen-

tial time, the best parallel time, and the relative speedup. Several interesting observations emerge from these data:

- **Scaling with image size and threads:** OpenMP provides moderate speedup compared to the sequential CPU implementation, especially for small to medium images. For example, on a $256 \times 256$ image with a $3 \times 3$ structuring element, OpenMP only reduces execution time from 31.57 ms to 26.75 ms. However, OpenMP is consistently slower than the best CUDA configuration, and its benefit becomes relatively smaller as image size increases.

- **GPU memory impact:** Shared memory generally provides the best performance for larger images and structuring elements, due to reduced memory latency and improved parallel throughput. For smaller structuring elements and images, the performance difference between global, constant, and shared memory is minimal, highlighting the effectiveness of CUDA caching mechanisms. For example, on a $5 \times 5$ structuring element with a $256 \times 256$ image, global, constant, and shared memory implementations perform similarly (0.889–0.933 ms), whereas on larger images like $1024 \times 1024$, shared memory consistently outperforms the other memory models.

- **Speedup trends:** GPU speedups are huge, often exceeding 100x compared to sequential CPU execution. The speedup increases with both image size and structuring element size; larger workloads and more compute-intensive operations benefit most from GPU acceleration. For instance, for a $3 \times 3$ structuring element on a $512 \times 512$ image, the speedup is roughly 141x, while for a $5 \times 5$ structuring element on a $1024 \times 1024$ image, it reaches approximately 240x. This demonstrates the massive advantage of GPU offloading for high-resolution processing tasks.

- **Algorithmic vs hardware limits:** Sequential execution time grows rapidly with both structuring element size and image size, consistent with the quadratic complexity of the morphological operation. CPU parallelization provides moderate improvement, but significant performance gains require GPU offloading. CUDA with shared memory consistently delivers the best results, emphasizing that hardware-aware optimization is essential to achieve high performance.

- **Critical insight:** Even for small images, GPU acceleration is substantially faster than CPU parallelization. For larger images, the advantage grows dramatically, making OpenMP a suboptimal choice for high-resolution or computationally intensive processing tasks. CPU parallelism may be useful for medium workloads, but for large-scale image processing, GPU memory optimization and massive parallelism are crucial.

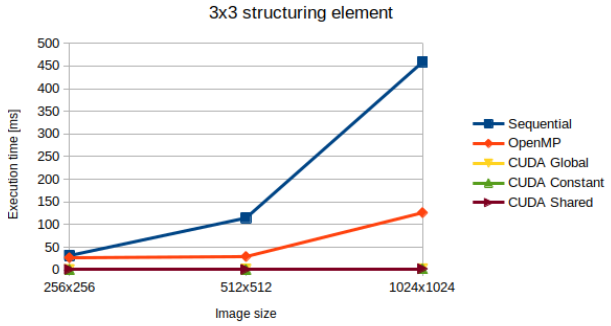


Figure 3: Execution times - 3x3 structuring element.

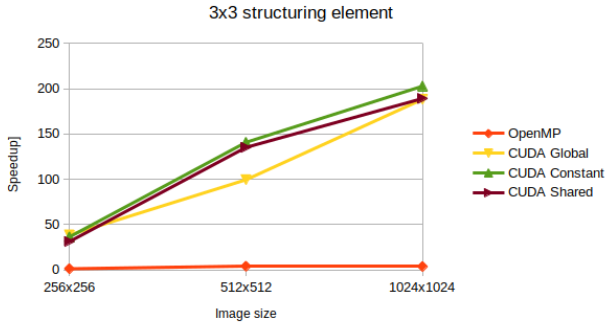


Figure 4: Speedups - 3x3 structuring element.

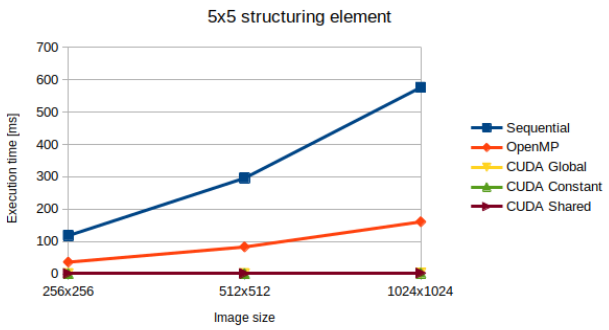


Figure 5: Execution times - 5x5 structuring element.

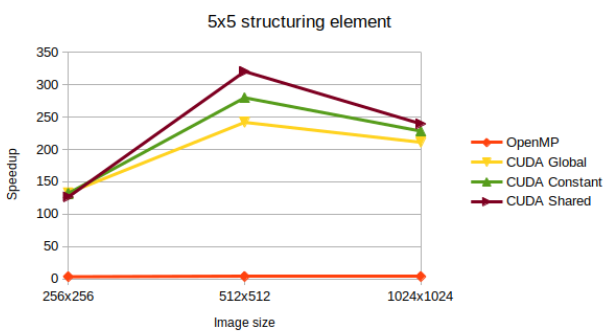


Figure 6: Speedups - 5x5 structuring element.

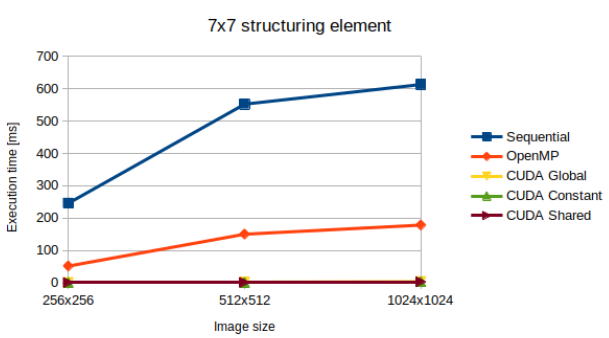


Figure 7: Execution times - 7x7 structuring element.

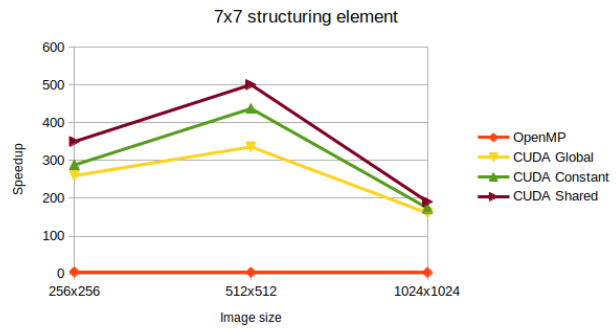


Figure 8: Speedups - 7x7 structuring element.

The following tables show the best execution times and the relative speedups for the gradient operation with various structuring element size. CUDA shared memory was chosen as the parallel variant because achieves the best results.

| Img size | Sequential | Parallel | Best speedup |
|---|---|---|---|
| 256x256 | 31.57ms | 0.82ms | 38.50x |
| 512x512 | 114.52ms | 0.81ms | 140.80x |
| 1024x1024 | 458.74ms | 2.26ms | 202.70x |

Table 10: Gradient operation with 3x3 structuring element

| Img size | Sequential | Parallel | Best speedup |
|---|---|---|---|
| 256x256 | 118.08ms | 0.89ms | 132.80x |
| 512x512 | 295.84ms | 0.92ms | 320.20x |
| 1024x1024 | 576.38ms | 2.41ms | 239.60x |

Table 11: Gradient operation with 5x5 structuring element

| Img size | Sequential | Parallel | Best speedup |
|---|---|---|---|
| 256x256 | 245.99ms | 0.70ms | 350.30x |
| 512x512 | 552.25ms | 1.10ms | 501.00x |
| 1024x1024 | 612.95ms | 3.21ms | 190.80x |

Table 12: Gradient operation with 7x7 structuring element

## 4 Conclusions

In this work, a morphological-based image processing application was analyzed with the objective of evaluating its performance across different execution paradigms. The local and data-parallel

nature of morphological operations makes it a representative and meaningful case study for assessing both CPU and GPU computing models.
The implemented solutions illustrate how the same processing workflow can be expressed using a sequential CPU approach, shared-memory parallelism with OpenMP, and GPU acceleration through CUDA. While the OpenMP implementation provides moderate improvements for small to medium images, substantial performance gains are achieved only with CUDA, particularly when using shared memory for larger images and more compute-intensive structuring elements. This demonstrates that minor modifications to the application structure can enable massive speedups, with GPU memory optimization playing a key role in achieving high performance. Overall, these results confirm the effectiveness of GPU acceleration for high-resolution and computationally heavy image processing, while highlighting the limitations of CPU parallelism in such contexts. The gradient operation thus represents a valuable benchmark for exploring parallel computing techniques, providing a solid foundation for further performance analysis and optimization.

## References


[1] Wikipedia, *Mathematical morphology*. https://en.wikipedia.org/wiki/Mathematical_morphology

[2] Towards Data Science, *Understanding Morphological Image Processing and Its Operations* https://towardsdatascience.com/understanding-morphological-image-processing-and-its-operations-7bcf1ed11756/

[3] CV Explained, *Morphological Operations*. https://cvexplained.wordpress.com/2020/05/18/morphological-operations/

[4] CUDA Documentation, *Get started with CUDA*. https://developer.nvidia.com/cuda

[5] OpenMP Documentation, *OpenMP reference guides*. https://www.openmp.org/resources/refguides/

[6] Rafael C. Gonzalez and Richard E. Woods, *Digital Image Processing*, 3rd Edition, Prentice Hall, 2008.

[7] S. Umbaugh, *Computer Vision and Image Processing: A Practical Approach Using CVIPtools*, Prentice Hall, 1998.

[8] Shared memory: https://devblogs.nvidia.com/using-shared-memory-cuda-cc/

[9] CUDA Programming Guide: https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#shared-memory

[10] NVIDIA Developer Blog, *Efficient Morphological Operations on GPUs*. https://developer.nvidia.com/blog/

[11] NVIDIA Corporation, *CUDA SHARED MEMORY*

[12] William K. Pratt, *Digital Image Processing*, Second Edition, 1991, Sun Microsystems, California